BANK PROJECT — MASTER PROJECT MANAGEMENT & DEVELOPMENT ROADMAP

Purpose of this document:
This is the master context and development roadmap for the Bank Management System microservices project.

Give this entire document to a new ChatGPT conversation when continuing the project.
The assistant should read the **CURRENT STATUS**, **ARCHITECTURE**, **COMPLETED STEPS**, and especially **NEXT STEP** before suggesting or writing code.

Important: Do not redesign the project architecture unless explicitly requested by the user. Continue from the existing implementation.

---

1. PROJECT NAME

Bank Management System

The project is being developed as a **Java Spring Boot microservices application**.

The main purpose is to implement a bank-management backend while learning and practicing:

- Spring Boot
- REST APIs
- Microservices
- Service-to-service communication
- JWT authentication
- Spring Security
- NGINX
- APISIX API Gateway
- Eureka Service Discovery
- Apache Kafka
- Event-driven communication
- Database persistence
- Load balancing
- Exception handling
- API testing with cURL/Postman
- Docker/Docker Compose

---

2. DEVELOPMENT PHILOSOPHY

The project is being built incrementally.

Do NOT jump directly to a completely different production architecture.

The existing project structure and implementation decisions should be preserved.

When adding functionality:

1. Understand the existing code.
2. Follow the existing package structure.
3. Reuse existing DTO/entity/service/controller patterns.
4. Do not introduce unnecessary frameworks.
5. Do not introduce Spring Cloud OpenFeign.
6. Use the already-selected communication approach.
7. Test each layer before moving to the next.
8. Explain why each class/file is needed.
9. Keep the implementation beginner-friendly and consistent.

---

3. MOST IMPORTANT ARCHITECTURE DECISIONS

3.1 Microservices

The project contains these services:

```
User Service
Account Service
Transaction Service
Notification Service
Auth Service
```

Infrastructure:

```
NGINX
APISIX
Eureka
Kafka
MySQL
Docker
```

---

4. FIXED PORT MAP

These ports are important and should NOT be changed unless the user explicitly requests it.

Component	Port	Purpose
User Service	8080	User management
Account Service	8081	Main account service
Account Service Instance 2	8085	Load-balancing practice only
Transaction Service	8082	Transactions
Notification Service	8083	Notifications
Auth Service	8084	Authentication/JWT
NGINX	8090	Reverse proxy/load balancing
NGINX internal	8080	Container listening port
Kafka host	19092	Kafka broker access
Kafka internal/controller	9093	Kafka controller
APISIX	9080	API Gateway
APISIX Admin API	9180	APISIX configuration
APISIX sample web1	9081	APISIX practice
APISIX sample web2	9082	APISIX practice
Eureka	8761	Service discovery
etcd	2379	APISIX configuration storage

Critical clarification

8081 is the main Account Service.

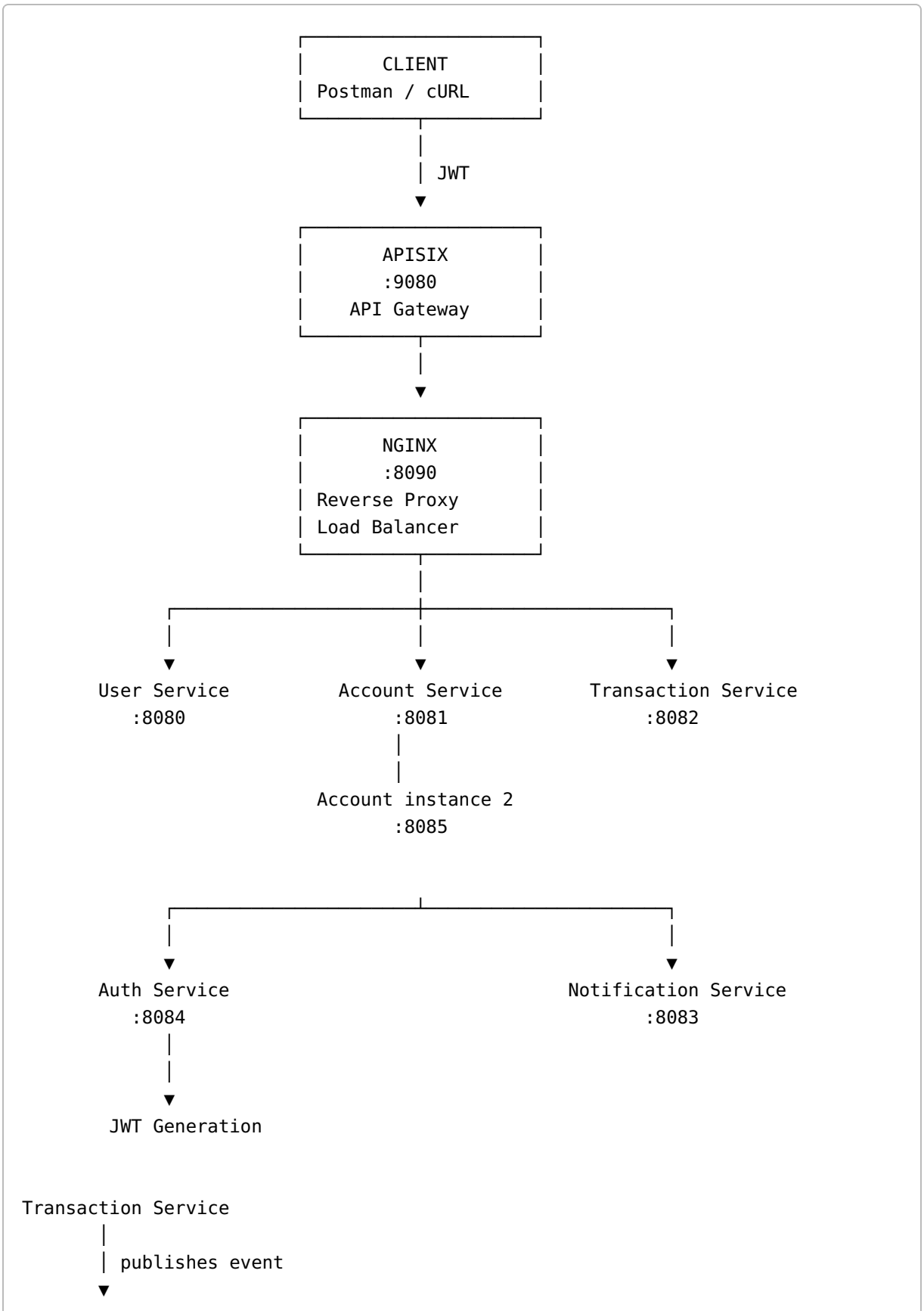
8085 is only a second Account Service instance used for NGINX load-balancing practice.

Do not treat 8085 as the primary Account Service.

---

5. HIGH-LEVEL SYSTEM ARCHITECTURE

Current intended architecture:


CLIENT
Postman / cURL
JWT
APISIX
:9080
API Gateway
NGINX
:8090
Reverse Proxy
Load Balancer
User Service
:8080
Account Service
:8081
Transaction Service
:8082
Account instance 2
:8085
Auth Service
:8084
Notification Service
:8083
JWT Generation
Transaction Service
publishes event

```
     Kafka
    :19092
       │
       │ consumes event
       ▼
Notification Service
       │
       ▼
     MySQL
```

---

6. SERVICE RESPONSIBILITIES

6.1 User Service — 8080

Responsible for:

- User registration
- Get user
- Get all users
- Update user
- Delete user
- User database persistence
- User validation
- User lookup required by Auth Service

Main endpoint structure:

```
/users/register
/users/{userId}
/users
```

6.2 Account Service — 8081

Responsible for:

- Creating accounts
- Getting account details
- Updating account balance
- Account-related operations
- Account database persistence

This is the **main Account Service**.

Second instance:

```
8085
```

exists only to demonstrate NGINX load balancing.

6.3 Transaction Service — 8082

Responsible for:

- Deposit
- Withdraw
- Transfer
- Transaction persistence
- Calling Account Service to update balances
- Publishing transaction-completed Kafka events

Current transaction flow:

```
Transaction Request
       │
       ▼
Transaction Service
       │
       ├── Get Account
       │
       ├── Update Account Balance
       │
       ├── Save Transaction
       │
       └── Publish Kafka Event
                   │
                   ▼
                Kafka
```

Communication with Account Service uses `RestClient`.

Do NOT introduce OpenFeign.

6.4 Notification Service — 8083

Responsible for:

- Consuming transaction events from Kafka
- Creating notifications
- Getting notifications by account
- Getting notification by notification ID
- Marking notification as read
- Future unread notification operations
- Future mark-all-as-read operation

Notification flow:

```
Transaction Service
       |
       | Kafka event
       ▼
     Kafka
       |
       ▼
Notification Consumer
       |
       ▼
Notification Service
       |
       ▼
MySQL
```

6.5 Auth Service — 8084

Responsible for:

- Login
- Validating credentials
- Generating JWT
- Returning JWT to client

Flow:

```
Client
  |
```

```
  | email + password
  ▼
Auth Service
  |
  ▼
User Service
  |
  | user/authentication information
  ▼
Auth Service
  |
  | generate JWT
  ▼
Client
```

The Auth Service owns:

```
AuthUserResponse
LoginRequest
LoginResponse
JwtService
```

Do not duplicate `AuthUserResponse` unnecessarily in User Service.

---

7. DATABASE

The project uses MySQL.

Services have database persistence for their respective data.

Important existing notification database observations:

```
notificationId
transactionId
accountNumber
message
status
createdAt
```

Notification statuses include:

```
UNREAD
READ
```

---

8. USER SERVICE — COMPLETED IMPLEMENTATION

8.1 User Controller

Current endpoints include:

```
POST   /users/register
GET    /users/{userId}
GET    /users
PUT    /users/{userId}
DELETE /users/{userId}
```

8.2 User Service Interface

Current operations:

```
UserResponse registerUser(UserRequest userRequest);

UserResponse getUserById(Long userId);

List<UserResponse> getAllUsers();

UserResponse updateUser(
    Long userId,
    UserRequest userRequest
);

void deleteUser(Long userId);
```

8.3 User Entity

Current fields:

```
userId
name
email
password
phoneNumber
```

Validation includes:

```
@NotBlank
@Email
@Pattern
```

Email and phone number are unique.

8.4 User Repository

Current important methods:

```
boolean existsByEmail(String email);

boolean existsByPhoneNumber(String phoneNumber);

Optional<UserEntity> findByEmail(String email);
```

8.5 User Response

The public UserResponse contains:

```
userId
name
email
phoneNumber
```

Password is not returned in the normal UserResponse.

---

9. ACCOUNT SERVICE — COMPLETED / CURRENT SECURITY WORK

Account Service has:

```
security/
    JwtService.java
    JwtFilter.java
    SecurityConfig.java
```

Spring Security dependency was required and added.

---

10. JWT ARCHITECTURE

Auth Service generates the JWT.

Other protected services validate the JWT.

Current approach:

```
Auth Service
     |
     | generates JWT
     ▼
   Client
     |
     | Authorization: Bearer JWT
     ▼
Account / Transaction / Notification
     |
     | validate JWT locally
     ▼
Request accepted
```

A service does NOT need to copy the user from User Service just to validate the JWT.

---

11. IMPORTANT JWT CONCEPT

`UsernamePasswordAuthenticationToken` does NOT replicate the user.

It creates an in-memory Spring Security `Authentication` object for the current HTTP request.

Flow:

```
JWT
 │
 ▼
JwtFilter
 │
 ├── validate token
 │
 ├── extract email
 │
 ▼
UsernamePasswordAuthenticationToken
 │
 ▼
SecurityContextHolder
 │
 ▼
Spring Security knows:
"This request is authenticated"
```

This is necessary because JWT validation alone does not automatically tell Spring Security that the current request is authenticated.

---

12. ACCOUNT JWT FILTER

The Account Service JWT filter should:

1. Read `Authorization`.
2. Check `Bearer`.
3. Extract token.
4. Validate JWT.
5. Extract email.
6. Create `UsernamePasswordAuthenticationToken`.
7. Put Authentication into `SecurityContextHolder`.
8. Continue the filter chain.

Conceptual implementation:

```
String authorizationHeader =
        request.getHeader("Authorization");
```

```
if (authorizationHeader == null
        || !authorizationHeader.startsWith("Bearer ")) {

    // 401
}

String token =
        authorizationHeader.substring(7);

if (!jwtService.isTokenValid(token)) {

    // 401
}

String email =
        jwtService.extractEmail(token);

UsernamePasswordAuthenticationToken authentication =
        new UsernamePasswordAuthenticationToken(
                email,
                null,
                Collections.emptyList()
        );

SecurityContextHolder
        .getContext()
        .setAuthentication(authentication);

filterChain.doFilter(request, response);
```

---

13. SPRING SECURITY CONFIGURATION

Current intended configuration:

```
@Configuration
public class SecurityConfig {

    private final JwtFilter jwtFilter;

    public SecurityConfig(JwtFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .addFilterBefore(
                jwtFilter,
                UsernamePasswordAuthenticationFilter.class
            );

        return http.build();
    }
}
```

---

14. WHY 403 OCCURRED

The earlier problem was that the JWT could be valid, but Spring Security did not know that the current request was authenticated.

Without:

```
SecurityContextHolder
    .getContext()
    .setAuthentication(authentication);
```

Spring Security can see:

```
No authenticated user
       |
       ▼
anyRequest().authenticated()
       |
       ▼
Access denied
```

With it:

```
Valid JWT
   |
   ▼
Authentication object
   |
   ▼
SecurityContext
   |
   ▼
Authenticated request
   |
   ▼
Controller
```

---

15. AUTH SERVICE — COMPLETED

Auth Service runs on:

```
8084
```

Current properties include:

```
spring.application.name=BM_AuthService
server.port=8084
user-service.url=http://localhost:8080
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

---

16. AUTH REST CLIENT

Auth Service uses `RestClient` to communicate with User Service.

Current configuration concept:

```
@Configuration
public class RestClientConfig {

    @Bean
    public RestClient restClient(
```

```
            @Value("${user-service.url}") String userServiceUrl) {

        return RestClient.builder()
                .baseUrl(userServiceUrl)
                .build();
    }
}
```

No OpenFeign.

---

17. AUTH LOGIN FLOW

Current implementation:

```
LoginRequest
    |
    | email + password
    ▼
AuthServiceImpl
    |
    ▼
UserServiceClient
    |
    ▼
User Service
    |
    ▼
AuthUserResponse
    |
    ▼
Compare password
    |
    ▼
JwtService.generateToken()
    |
    ▼
LoginResponse
```

Successful login was already tested.

Response contains:

```
message
token
userId
email
```

Do not expose or unnecessarily repeat the actual JWT token.

---

18. AUTH EXCEPTION HANDLING

Custom exception:

```
BM_AuthService.exception.InvalidCredentialsException
```

Important:

Do NOT import the Apache HTTP client version of:

```
InvalidCredentialsException
```

Use the project's own:

```
BM_AuthService.exception.InvalidCredentialsException
```

Global exception handling exists through:

```
GlobalHandlerException
```

---

19. AUTH JWT SERVICE

Auth Service uses JJWT 0.12.x style APIs.

Important implementation concepts:

```
SecretKey
Keys.hmacShaKeyFor(...)
```

and:

```
Jwts.parser()
    .verifyWith(getSigningKey())
    .build()
    .parseSignedClaims(token)
```

The JWT contains:

```
subject = email
userId claim
issuedAt
expiration
```

The other protected services must use the same signing configuration for validation.

---

20. TRANSACTION SERVICE — COMPLETED BUSINESS LOGIC

Transaction Service runs on:

```
8082
```

It contains:

```
TransactionRepository
TransactionService
TransactionServiceImpl
TransactionController
TransactionRequest
TransactionResponse
AccountResponse
TransactionCompletedEvent
TransactionEventProducer
```

---

21. TRANSACTION BUSINESS FLOW

Deposit

```
POST /transactions/deposit
       |
       ▼
Get Account
       |
       ▼
Update Account Balance
       |
       ▼
Create Transaction
       |
       ▼
Save Transaction
       |
       ▼
Publish transaction-completed
       |
       ▼
Kafka
```

---

22. WITHDRAW

```
POST /transactions/withdraw
       |
       ▼
Get Account
       |
       ▼
Check balance/business rules
       |
       ▼
Update Account Balance
       |
       ▼
Save Transaction
       |
```

```
      ▼
Publish Kafka event
```

---

23. TRANSFER

Current implementation creates the sender-side transaction/event.

Conceptually:

```
Sender Account
      |
      ▼
Validate
      |
      ▼
Update Sender Balance
      |
      ▼
Save Transaction
      |
      ▼
Publish transaction-completed
```

Receiver handling should be reviewed later if the project requirements require a separate receiver transaction/notification.

Do not redesign this unless requested.

---

24. TRANSACTION → ACCOUNT COMMUNICATION

Transaction Service uses:

```
RestClient
```

It communicates with:

```
/accounts/{accountNumber}
```

and:

```
PATCH /accounts/{accountNumber}/balance
```

with:

```
amount
operation
```

No OpenFeign.

---

25. KAFKA — COMPLETED

Kafka is running through Docker.

Current host access:

```
localhost:19092
```

Kafka container:

```
kafka-broker
```

Kafka controller:

```
9093
```

---

26. KAFKA DOCKER CONFIGURATION

The working Kafka setup is conceptually:

```
kafka:
  image: apache/kafka:latest
  container_name: kafka-broker
  ports:
```

```
    - "19092:9092"
    - "9093:9093"
```

Important configuration includes:

```
KAFKA_NODE_ID=1
KAFKA_PROCESS_ROLES=broker,controller
KAFKA_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER
KAFKA_CONTROLLER_QUORUM_VOTERS=1@localhost:9093
```

Kafka was previously broken but was deleted/recreated and successfully fixed.

Do not unnecessarily replace the working Kafka configuration.

---

27. KAFKA TOPIC

Current topic:

```
transaction-completed
```

---

28. TRANSACTION EVENT PRODUCER

Current producer:

```
@Service
public class TransactionEventProducer {

    private static final String TOPIC =
            "transaction-completed";

    private final KafkaTemplate<
            String,
            TransactionCompletedEvent> kafkaTemplate;

    public TransactionEventProducer(
            KafkaTemplate<
                    String,
                    TransactionCompletedEvent>
```

```
                    kafkaTemplate) {

        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishTransactionCompleted(
            TransactionCompletedEvent event) {

        kafkaTemplate.send(
                TOPIC,
                String.valueOf(event.getTransactionId()),
                event
        );
    }
}
```

---

29. TRANSACTION EVENT DATA

The Transaction Service event contains:

```
transactionId
accountNumber
amount
transactionType
status
receiverAccountNumber
transactionTime
balanceAfterTransaction
```

---

30. KAFKA TEST — COMPLETED

A direct Transaction Service deposit was successfully tested.

Example successful transaction contained:

```
transactionId
accountNumber
amount
```

```
transaction type = DEPOSIT
status = SUCCESS
```

Kafka was previously producing:

```
Send failed
```

but the Kafka configuration was corrected and the transaction event flow subsequently worked.

---

31. NOTIFICATION SERVICE — COMPLETED

Notification Service runs on:

```
8083
```

It contains:

```
Notification
NotificationRepository
NotificationService
NotificationServiceImpl
NotificationController
TransactionCompletedEvent
TransactionEventConsumer
```

---

32. NOTIFICATION ENTITY

Fields:

```
notificationId
transactionId
accountNumber
message
status
createdAt
```

Status:

```
UNREAD
READ
```

---

33. NOTIFICATION REPOSITORY

Current important methods:

```
List<Notification>
findByAccountNumberOrderByCreatedAtDesc(
    Long accountNumber
);

boolean existsByTransactionId(
    Long transactionId
);
```

The `existsByTransactionId()` check makes notification creation idempotent.

This prevents duplicate notifications for the same transaction event.

---

34. NOTIFICATION SERVICE INTERFACE

Current implemented operations:

```
void createNotification(
    TransactionCompletedEvent event
);

List<NotificationResponse>
getNotificationsByAccountNumber(
    Long accountNumber
);

NotificationResponse getNotificationById(
    Long notificationId
);

void markAsRead(
```

```
    Long notificationId
);
```

Planned additional operations:

```
getUnreadNotificationsByAccountNumber(
    Long accountNumber
);

markAllAsRead(
    Long accountNumber
);
```

---

35. NOTIFICATION CONTROLLER

Current endpoints:

```
GET /notifications/account/{accountNumber}

GET /notifications/{notificationId}

PUT /notifications/{notificationId}/read
```

Future endpoints should be added for:

```
GET unread notifications
PUT mark all as read
```

---

36. KAFKA NOTIFICATION CONSUMER

Current consumer:

```
@Service
public class TransactionEventConsumer {

    private final NotificationService notificationService;

    public TransactionEventConsumer(
```

```
            NotificationService notificationService) {

        this.notificationService =
                notificationService;
    }

    @KafkaListener(
        topics = "transaction-completed",
        groupId = "notification-service"
    )
    public void consume(
            TransactionCompletedEvent event) {

        notificationService
            .createNotification(event);
    }
}
```

---

37. NOTIFICATION KAFKA DTO

Current consumer DTO contains:

```
transactionId
accountNumber
amount
transactionType
status
receiverAccountNumber
```

Transaction Service currently sends additional fields:

```
transactionTime
balanceAfterTransaction
```

Jackson currently allows the consumer to ignore unknown event fields.

This should be reviewed later so both event DTOs have the same schema.

---

38. NOTIFICATION TESTING — COMPLETED

Direct notification retrieval was successfully tested.

Example:

```
GET http://localhost:8083/notifications/1
```

worked.

Mark-as-read was also successfully tested:

```
PUT http://localhost:8083/notifications/1/read
```

The notification changed from:

```
UNREAD
```

to:

```
READ
```

Earlier `[]` results were caused by using the wrong account number / Postman tab/request context rather than a database failure.

---

39. NGINX

NGINX is used for:

- Reverse proxy
- Service routing
- Load balancing
- Account Service two-instance load balancing

Host:

```
localhost:8090
```

NGINX container listens internally on:

```
8080
```

---

40. NGINX CURRENT ROUTES

Intended routes:

```
/users/          → User Service
/accounts/       → Account Service
/transactions/   → Transaction Service
/auth/           → Auth Service
/notifications/  → Notification Service
```

---

41. NGINX ACCOUNT LOAD BALANCING

Account upstream:

```
upstream account_service {
    server host.docker.internal:8081;
    server host.docker.internal:8085;
}
```

This is intentional.

Requests to:

```
/accounts/...
```

can be distributed between:

```
8081
8085
```

This is primarily for learning/practice of load balancing.

---

42. IMPORTANT NGINX BUG TO FIX

The current NGINX configuration has the Auth and Notification ports reversed.

Current incorrect mapping:

```
auth_service        → 8083
notification_service → 8084
```

Correct mapping must be:

```
auth_service         → 8084
notification_service → 8083
```

Correct:

```
upstream auth_service {
    server host.docker.internal:8084;
}

upstream notification_service {
    server host.docker.internal:8083;
}
```

This must be fixed before final APISIX integration testing.

---

43. NGINX ROUTING ARCHITECTURE

Intended:

```
APISIX
  │
  ▼
NGINX :8090
  │
  ├── /users/          → 8080
  ├── /accounts/       → 8081 / 8085
  ├── /transactions/   → 8082
```

```
  ├── /auth/           → 8084
  └── /notifications/  → 8083
```

---

44. APISIX

APISIX is being used as the API Gateway.

Important ports:

```
9080 = API traffic
9180 = Admin API
```

APISIX configuration uses the Admin API.

Admin API requires:

```
X-API-KEY
```

Do not invent the actual API key.

Use the API key configured in the user's APISIX setup.

---

45. APISIX INTENDED ROLE

APISIX should be the public API gateway.

Conceptually:

```
Client
  │
  ▼
APISIX :9080
  │
  ▼
NGINX :8090
  │
  ▼
Microservices
```

The project intentionally practices both:

```
APISIX
```

and:

```
NGINX
```

instead of replacing one with the other.

---

46. APISIX ROUTES STILL REQUIRED

Routes need to be configured for:

```
/auth/*
/notifications/*
```

and existing service routes should be reviewed.

Preferred architecture:

```
APISIX
    ↓
NGINX
    ↓
Correct backend service
```

---

47. EUREKA

Eureka runs on:

```
8761
```

Services use:

```
eureka.client.service-url.defaultZone=
http://localhost:8761/eureka/
```

Eureka is being used for service discovery practice.

Do not suddenly replace the architecture with Kubernetes service discovery or another discovery system.

---

48. CURRENT SECURITY STATUS

Auth Service

JWT generation:

```
DONE
```

Login:

```
DONE / TESTED
```

Account Service

JWT validation:

```
IMPLEMENTED / BEING TESTED
```

Spring Security:

```
ADDED
```

JWT filter:

```
IMPLEMENTED
```

SecurityContext authentication:

```
IMPLEMENTED
```

Transaction Service

JWT security:

```
NEXT IMPLEMENTATION STEP
```

Need:

```
spring-boot-starter-security
JwtService
JwtFilter
SecurityConfig
```

Then test:

```
without JWT → 401
with valid JWT → transaction executes
```

Notification Service

JWT security:

```
NOT YET IMPLEMENTED
```

Need the same basic security structure after Transaction Service is completed.

User Service

JWT security:

```
NOT YET DECIDED / TO BE REVIEWED
```

Determine which User Service endpoints should remain public:

```
POST /users/register
```

and which should require JWT.

Likely:

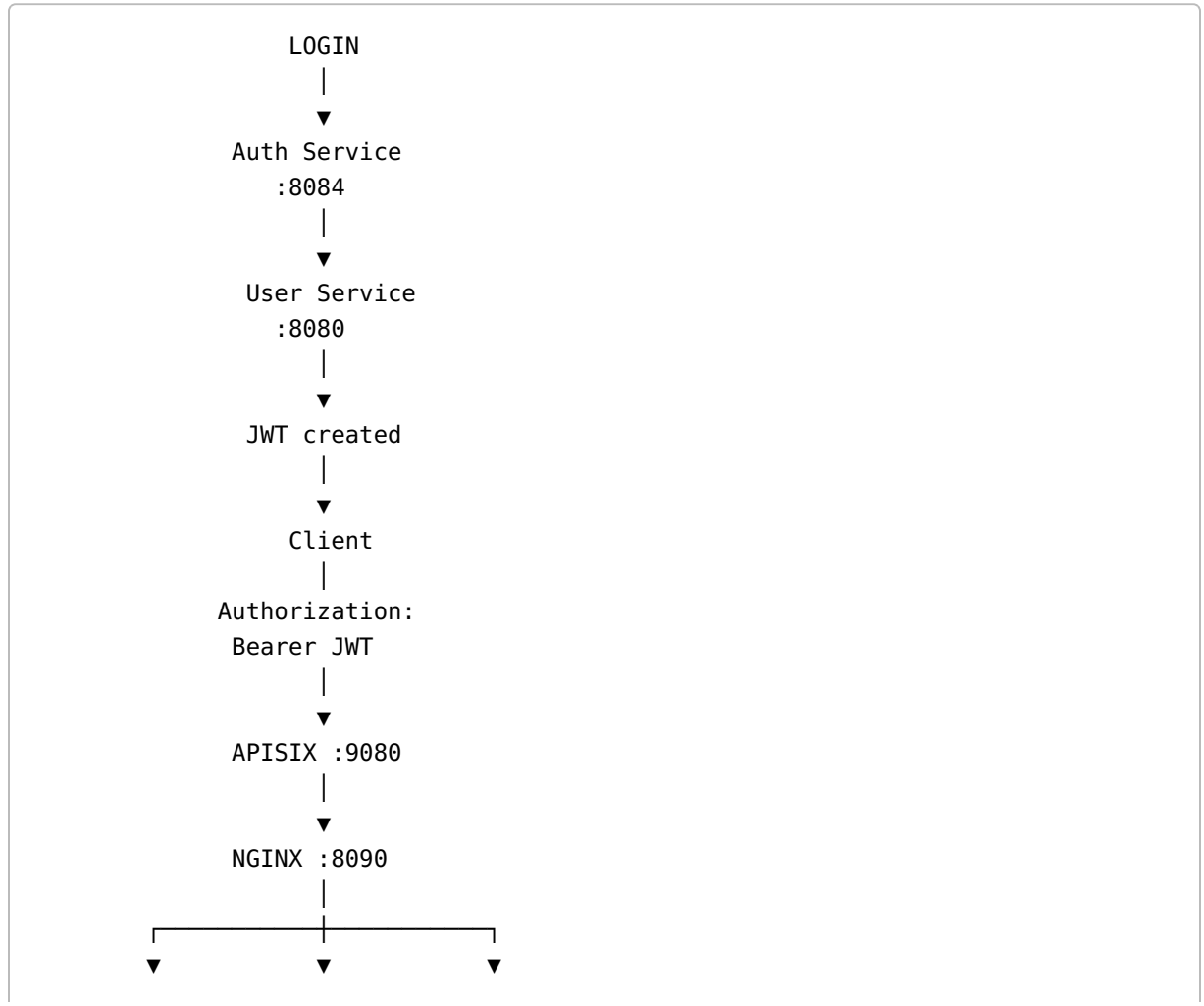
```
/register → public
other user management → authenticated
```

But confirm against project requirements before changing.

---

49. COMPLETE SECURITY FLOW

Final intended flow:



```
    Account    Transaction   Notification
     :8081        :8082         :8083
        |           |             |
        ▼           ▼             ▼
    JwtFilter   JwtFilter     JwtFilter
        |           |             |
        ▼           ▼             ▼
     Validate   Validate       Validate
       JWT        JWT           JWT
```

---

50. COMPLETE TRANSACTION + NOTIFICATION FLOW

Final intended flow:

```
Client
  |
  | JWT + transaction request
  ▼
APISIX
  |
  ▼
NGINX
  |
  ▼
Transaction Service
  |
  ├── Validate JWT
  |
  ├── Get Account
  |
  ├── Update Balance
  |
  ├── Save Transaction
  |
  └── Publish transaction-completed
            |
            ▼
          Kafka
            |
            ▼
    Notification Consumer
            |
```

```
          ▼
    Notification Service
          │
          ▼
        MySQL
```

---

51. DEVELOPMENT PHASES

The complete project should be developed in the following phases.

PHASE 1 — Basic Microservices

Completed

```
1. Create User Service
2. Create Account Service
3. Create Transaction Service
4. Create Notification Service
5. Create Auth Service
6. Configure ports
7. Configure MySQL
8. Create entities
9. Create repositories
10. Create DTOs
11. Create service interfaces
12. Create service implementations
13. Create controllers
```

Status:

```
COMPLETED
```

---

52. PHASE 2 — User Management

Completed

```
1. User registration
2. Get user by ID
```

```
3. Get all users
4. Update user
5. Delete user
6. Email uniqueness
7. Phone uniqueness
8. Validation
9. Password storage
```

Status:

```
COMPLETED
```

---

53. PHASE 3 — Account Management

Completed / Existing

```
1. Account creation
2. Account retrieval
3. Balance update
4. Account persistence
5. Account service REST API
```

Status:

```
MOSTLY COMPLETED
```

Need later:

```
Security verification
Load balancing verification
Gateway testing
```

---

54. PHASE 4 — Transaction Management

Completed

```
1. Deposit
2. Withdraw
3. Transfer
4. Account lookup through RestClient
5. Account balance update through RestClient
6. Transaction persistence
7. Transaction response
```

Status:

```
COMPLETED
```

---

55. PHASE 5 — Kafka Event Architecture

Completed

```
1. Kafka Docker setup
2. Kafka broker configuration
3. transaction-completed topic
4. TransactionEventProducer
5. TransactionCompletedEvent
6. Transaction Service event publishing
7. Notification Kafka consumer
8. Notification creation
```

Status:

```
COMPLETED
```

---

56. PHASE 6 — Notification System

Completed

```
1. Notification entity
2. Notification repository
3. Notification service
4. Notification service implementation
5. Kafka consumer
6. Create notification
7. Get notification by ID
8. Get notifications by account
9. Mark notification as read
10. Idempotency using transactionId
```

Remaining

```
11. Get unread notifications
12. Mark all notifications as read
13. Review event DTO consistency
14. Add JWT security
```

Status:

```
PARTIALLY COMPLETED
```

---

57. PHASE 7 — Authentication

Completed

```
1. Auth Service
2. LoginRequest
3. LoginResponse
4. AuthUserResponse
5. UserServiceClient
6. RestClient communication
7. Credential validation
8. InvalidCredentialsException
9. Global exception handling
10. JwtService
```

```
11. JWT generation
12. Login testing
```

Status:

```
COMPLETED
```

---

58. PHASE 8 — JWT SECURITY

Completed

```
1. Spring Security dependency
2. Account JwtService
3. Account JwtFilter
4. Account SecurityConfig
5. SecurityContext authentication
6. Understanding UsernamePasswordAuthenticationToken
```

Next

```
7. Transaction Service JWT
8. Notification Service JWT
9. User Service JWT where appropriate
10. End-to-end JWT testing
```

Status:

```
IN PROGRESS
```

---

59. PHASE 9 — NGINX

Completed / Existing

```
1. NGINX Docker setup
2. User routing
3. Account routing
4. Transaction routing
```

```
5. Auth routing
6. Notification routing
7. Account load balancing
```

Required

```
1. Fix Auth port
2. Fix Notification port
3. Test every route
4. Test Account 8081/8085 load balancing
5. Test JWT through NGINX
```

Status:

```
CONFIGURED — NEEDS FINAL VERIFICATION
```

---

60. PHASE 10 — APISIX

Existing

```
APISIX
9080
Admin API
9180
```

Required

```
1. Configure /auth/*
2. Configure /notifications/*
3. Review existing routes
4. Connect APISIX to NGINX
5. Test APISIX → NGINX
6. Test NGINX → Service
7. Test JWT through complete gateway chain
```

Status:

```
IN PROGRESS
```

61. PHASE 11 — EUREKA

Existing

```
Eureka :8761
```

Need to verify:

```
1. User registration
2. Account registration
3. Transaction service registration
4. Notification service registration
5. Auth service registration
6. Eureka dashboard
```

Then determine where Eureka is actually being used versus direct URLs.

Do not introduce unnecessary service-discovery changes.

---

62. PHASE 12 — END-TO-END TESTING

After security and gateway work:

Test the entire flow.

Test A — Register User

```
POST /users/register
```

Expected:

```
User created
```

Test B — Login

```
POST /auth/login
```

Expected:

```
JWT token
```

Test C — Account Request Without JWT

Expected:

```
401 Unauthorized
```

Test D — Account Request With JWT

Expected:

```
Account details
```

Test E — Deposit With JWT

```
POST /transactions/deposit
```

Expected:

```
Transaction successful
Balance updated
Kafka event published
Notification created
```

Test F — Notification Retrieval

```
GET /notifications/account/{accountNumber}
```

Expected:

```
Notification list
```

Test G — Mark Notification Read

```
PUT /notifications/{notificationId}/read
```

Expected:

```
status = READ
```

Test H — Gateway Test

Instead of directly calling:

```
localhost:8081
localhost:8082
localhost:8083
localhost:8084
```

test through:

```
localhost:9080
```

after APISIX is configured.

---

63. FINAL API FLOW

Final expected user journey:

```
1. Register
      ↓
2. Login
      ↓
3. Receive JWT
      ↓
4. Send JWT with API requests
      ↓
5. APISIX
      ↓
6. NGINX
      ↓
7. Correct Microservice
      ↓
8. JWT validation
      ↓
9. Business logic
      ↓
10. Kafka event where required
      ↓
11. Notification
      ↓
12. Database
```

---

64. CURRENT KNOWN ISSUES / TODO

These should be treated as the project's active backlog.

HIGH PRIORITY

TODO-01 — Transaction JWT Security

Add:

```
TransactionService
 └── security
      ├── JwtService.java
      ├── JwtFilter.java
      └── SecurityConfig.java
```

Then test:

```
No JWT → 401
Invalid JWT → 401
Valid JWT → transaction works
```

TODO-02 — Notification JWT Security

Add:

```
NotificationService
 └── security
      ├── JwtService.java
      ├── JwtFilter.java
      └── SecurityConfig.java
```

Then test notification endpoints.

TODO-03 — Fix NGINX Auth Port

Change:

```
8083
```

to:

```
8084
```

for Auth Service.

TODO-04 — Fix NGINX Notification Port

Change:

```
8084
```

to:

```
8083
```

for Notification Service.

TODO-05 — APISIX Auth Route

Configure:

```
/auth/*
```

TODO-06 — APISIX Notification Route

Configure:

```
/notifications/*
```

TODO-07 — Complete Gateway Testing

Test:

```
Client
 ↓
APISIX
 ↓
NGINX
 ↓
Service
```

---

65. MEDIUM PRIORITY TODO

```
TODO-08  Get unread notifications

TODO-09  Mark all notifications as read

TODO-10  Review TransactionCompletedEvent schema consistency
```

```
TODO-11  Secure User Service endpoints appropriately

TODO-12  Test Account 8081/8085 load balancing

TODO-13  Verify Eureka registrations

TODO-14  Test all APIs through APISIX

TODO-15  Test invalid/expired JWT

TODO-16  Test missing Authorization header

TODO-17  Test malformed Authorization header
```

---

66. FINAL HARDENING / CLEANUP TODO

After functionality is complete:

```
TODO-18  Improve exception handling

TODO-19  Improve validation

TODO-20  Review HTTP status codes

TODO-21  Review database constraints

TODO-22  Review transaction failure handling

TODO-23  Review Kafka failure handling

TODO-24  Review duplicate event handling

TODO-25  Review logging

TODO-26  Review configuration properties

TODO-27  Move JWT secret to configuration/environment

TODO-28  Dockerize/verify all services

TODO-29  Verify Docker networking
```

```
TODO-30  Complete README/documentation
```

---

67. IMPORTANT: DO NOT CHANGE THESE DECISIONS

Unless explicitly requested by the user:

Do NOT introduce:

```
Spring Cloud OpenFeign
```

Use:

```
RestClient
```

for current synchronous service-to-service calls.

Use:

```
Kafka
```

for transaction-completed event → notification.

Use:

```
JWT
```

for authentication.

Use:

```
Spring Security
```

for protected services.

Use:

```
NGINX
```

for reverse proxy and Account Service load balancing.

Use:

```
APISIX
```

as API Gateway.

Use:

```
Eureka
```

for service discovery practice.

---

68. HOW TO CONTINUE IN A NEW CHAT

When this document is supplied to ChatGPT, the assistant should NOT restart the project.

It should first determine:

```
CURRENT STATUS
        ↓
ACTIVE TODO
        ↓
NEXT STEP
```

Then continue from there.

The assistant should say something similar to:

```
I understand the project.

Current completed stage:
JWT/Auth + Kafka + Notification + NGINX are implemented.

Current active stage:
```

```
JWT security and API Gateway integration.

Next step:
Transaction Service JWT security.
```

Then provide only the necessary code/files for that step.

---

69. RULE FOR EACH DEVELOPMENT STEP

For every new implementation:

Step 1 — Explain the purpose

Explain:

```
Why are we adding this?
What problem does it solve?
```

Step 2 — Show project location

Example:

```
BM_TransactionService
└── src/main/java
    └── BM_TransactionService
        └── security
            └── JwtFilter.java
```

Step 3 — Give complete code

Do not provide fragmented pseudo-code when the user needs a file.

Step 4 — Explain important lines

Especially:

```
JWT
SecurityContext
RestClient
Kafka
```

```
NGINX
APISIX
```

Step 5 — Build

Tell the user what to run.

Step 6 — Test

Give exact cURL/Postman test.

Step 7 — Verify expected response

Example:

```
Without JWT → 401
With JWT → 200
```

Step 8 — Only then continue

Do not implement five unrelated features simultaneously.

---

70. CURRENT DEVELOPMENT POSITION

COMPLETED

```
[✓] User Service
[✓] Account Service
[✓] Transaction Service
[✓] Notification Service
[✓] Auth Service

[✓] User CRUD
[✓] Account operations
[✓] Deposit
[✓] Withdraw
[✓] Transfer

[✓] RestClient communication
[✓] Kafka
[✓] transaction-completed event
```

```
[✓] Notification consumer
[✓] Notification persistence

[✓] Login
[✓] JWT generation
[✓] Account JWT validation
[✓] Spring Security basics

[✓] NGINX setup
[✓] Account load-balancing setup

[✓] APISIX setup
[✓] Eureka setup
```

---

71. IN PROGRESS

```
[~] JWT security across all protected services

[~] NGINX final routing verification

[~] APISIX route configuration

[~] End-to-end gateway testing
```

---

72. EXACT NEXT STEP

NEXT STEP = TRANSACTION SERVICE JWT SECURITY

Do NOT skip directly to APISIX.

First implement:

```
BM_TransactionService
        │
        └── security
             ├── JwtService.java
             ├── JwtFilter.java
             └── SecurityConfig.java
```

Also add:

```
spring-boot-starter-security
```

Then test:

```
POST /transactions/deposit

WITHOUT JWT
        ↓
401 Unauthorized
```

and:

```
POST /transactions/deposit

WITH VALID JWT
        ↓
Transaction executes
        ↓
Account balance updates
        ↓
Transaction saved
        ↓
Kafka event
        ↓
Notification
```

---

73. AFTER TRANSACTION JWT

The exact sequence should be:

```
NEXT 1
Transaction Service JWT
        ↓
NEXT 2
Test Transaction JWT
        ↓
NEXT 3
Notification Service JWT
```

```
        ↓
NEXT 4
Test Notification JWT
        ↓
NEXT 5
Review User Service security
        ↓
NEXT 6
Fix NGINX Auth/Notification ports
        ↓
NEXT 7
Test NGINX routes
        ↓
NEXT 8
Configure APISIX Auth route
        ↓
NEXT 9
Configure APISIX Notification route
        ↓
NEXT 10
Test APISIX → NGINX → Services
        ↓
NEXT 11
Test complete JWT flow
        ↓
NEXT 12
Test Kafka + Notification end-to-end
        ↓
NEXT 13
Test Account load balancing
        ↓
NEXT 14
Verify Eureka
        ↓
NEXT 15
Exception/error handling
        ↓
NEXT 16
Configuration cleanup
        ↓
NEXT 17
Docker/network verification
        ↓
NEXT 18
Final project testing
        ↓
NEXT 19
Documentation / README
```

```
        ↓
PROJECT COMPLETE
```

---

74. CURRENT MASTER STATUS

```
================================================
 BANK MANAGEMENT SYSTEM
================================================

Architecture:
    Microservices

Authentication:
    JWT + Spring Security

API Gateway:
    APISIX

Reverse Proxy:
    NGINX

Service Discovery:
    Eureka

Event System:
    Apache Kafka

Synchronous Communication:
    RestClient

OpenFeign:
    NOT USED

Database:
    MySQL

------------------------------------------------
 SERVICES
------------------------------------------------

User Service:
    8080
    STATUS = Working
```

```
Account Service:
    8081
    STATUS = Working

Account Service Instance 2:
    8085
    STATUS = Load-balancing practice

Transaction Service:
    8082
    STATUS = Working
    JWT = NEXT

Notification Service:
    8083
    STATUS = Working
    JWT = NEXT AFTER TRANSACTION

Auth Service:
    8084
    STATUS = Working

-------------------------------------------------
 INFRASTRUCTURE
-------------------------------------------------

NGINX:
    8090
    STATUS = Configured
    ACTION = Fix Auth/Notification mapping

APISIX:
    9080
    STATUS = Running
    ACTION = Configure routes

APISIX Admin:
    9180

Kafka:
    19092
    STATUS = Working

Eureka:
    8761

etcd:
    2379
```

```
------------------------------------------------
 CURRENT DEVELOPMENT STAGE
------------------------------------------------

AUTHENTICATION + SECURITY + API GATEWAY

------------------------------------------------
 IMMEDIATE NEXT STEP
------------------------------------------------

TRANSACTION SERVICE JWT SECURITY

------------------------------------------------
 DO NOT RESTART
------------------------------------------------

The basic microservices,
Kafka,
Notification,
Auth,
JWT generation,
and Account security
are already implemented.

Continue from the active TODO list.

================================================
```

75. INSTRUCTION TO FUTURE CHATGPT

When this document is pasted into a new conversation:

Treat this document as the project's current source of truth.

Before answering:

1. Read the architecture.
2. Read the port map.
3. Read completed work.
4. Read known issues.
5. Read **EXACT NEXT STEP**.
6. Continue from that step.
7. Do not redesign the architecture.
8. Do not introduce OpenFeign.
9. Do not change ports.

10. Do not duplicate existing classes unnecessarily.
11. Keep the code consistent with the existing Bank Management System project.
12. If the user provides new code, treat that code as the newest implementation state and update the roadmap accordingly.
13. If a step is successfully tested, mark it as completed and move to the next step.
14. Always explain the purpose of new files before giving code.
15. Give exact testing commands after implementation.
16. Do not claim something works until the user has tested it or the available evidence confirms it.

---

76. PROJECT COMPLETION CRITERIA

The project should be considered complete only after:

```
[ ] User Service works
[ ] Account Service works
[ ] Transaction Service works
[ ] Notification Service works
[ ] Auth Service works

[ ] Login generates JWT
[ ] Account validates JWT
[ ] Transaction validates JWT
[ ] Notification validates JWT
[ ] Appropriate User endpoints validate JWT

[ ] Deposit works
[ ] Withdraw works
[ ] Transfer works

[ ] Account balance updates correctly

[ ] Kafka publishes transaction events
[ ] Notification consumes events
[ ] Notification is persisted

[ ] Notification can be retrieved
[ ] Notification can be marked READ
[ ] Unread notifications work
[ ] Mark-all-read works

[ ] NGINX routes all services correctly
[ ] Account load balancing works

[ ] APISIX routes all required APIs
```

```
[ ] APISIX → NGINX works

[ ] JWT works through APISIX
[ ] JWT works through NGINX

[ ] Eureka services register correctly

[ ] Invalid JWT returns 401
[ ] Missing JWT returns 401
[ ] Expired JWT returns 401

[ ] Service errors are handled properly
[ ] Kafka failure behavior reviewed
[ ] Database constraints reviewed

[ ] Docker setup verified

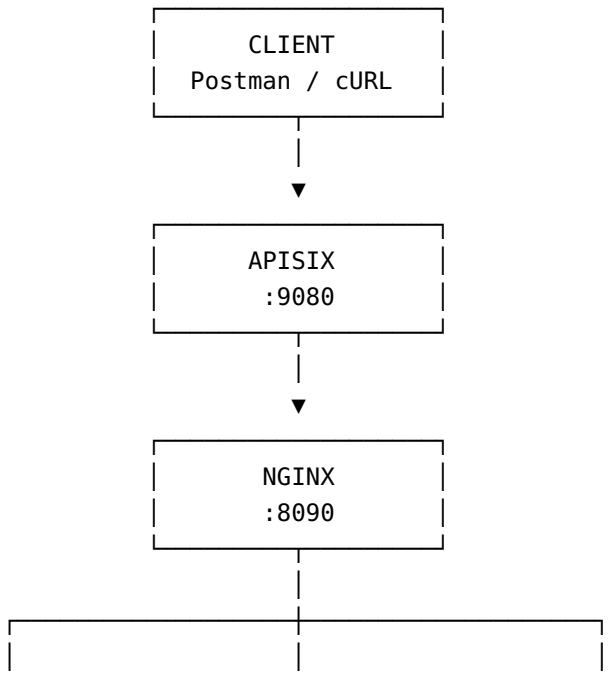
[ ] Complete end-to-end flow tested

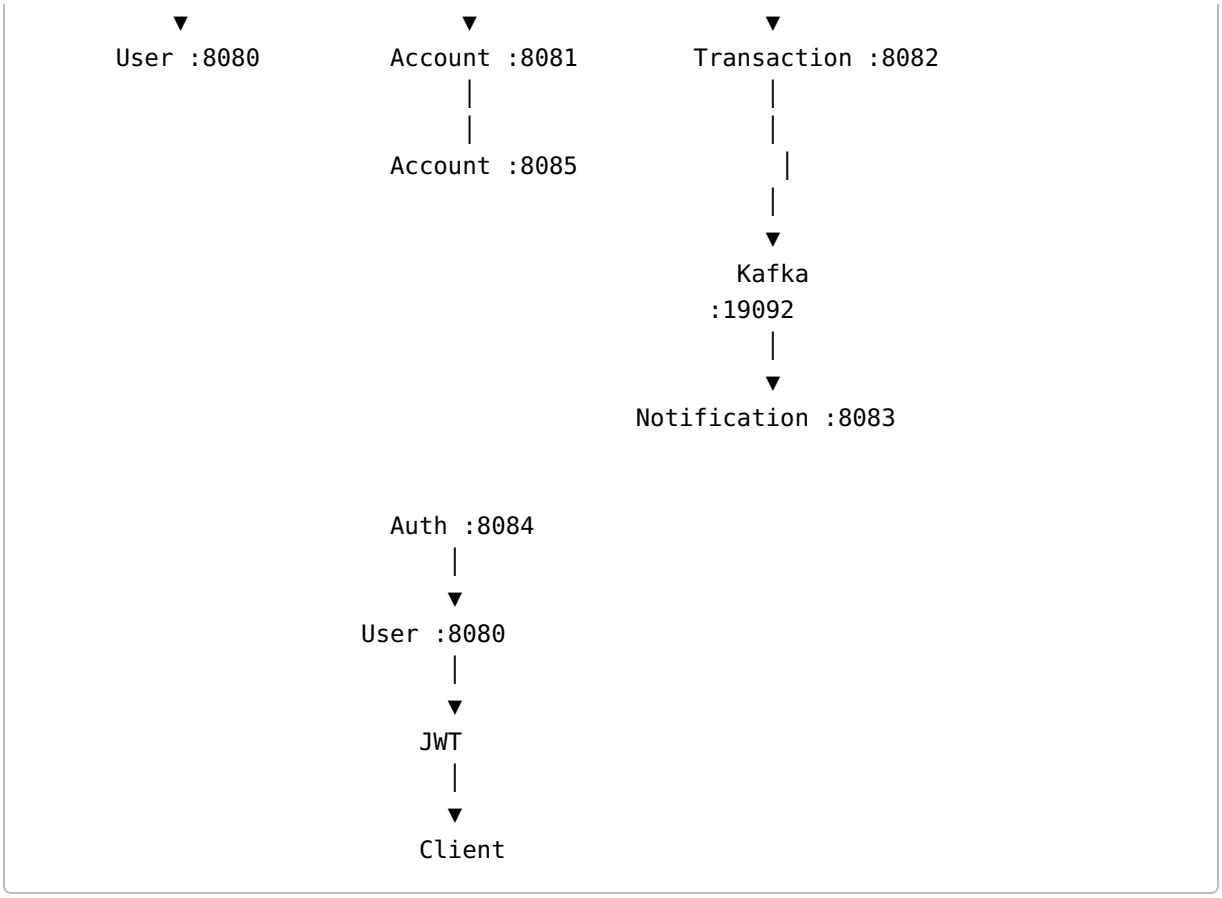
[ ] Project documentation completed
```

---

77. FINAL PROJECT FLOW

The finished system should demonstrate:





The ultimate business flow is:

```
REGISTER
   ↓
LOGIN
   ↓
JWT
   ↓
API REQUEST
   ↓
APISIX
   ↓
NGINX
   ↓
JWT VALIDATION
   ↓
BUSINESS OPERATION
   ↓
DATABASE
   ↓
KAFKA EVENT
```

```
   ↓
NOTIFICATION
   ↓
DATABASE
```

This is the master roadmap for continuing the Bank Management System project.